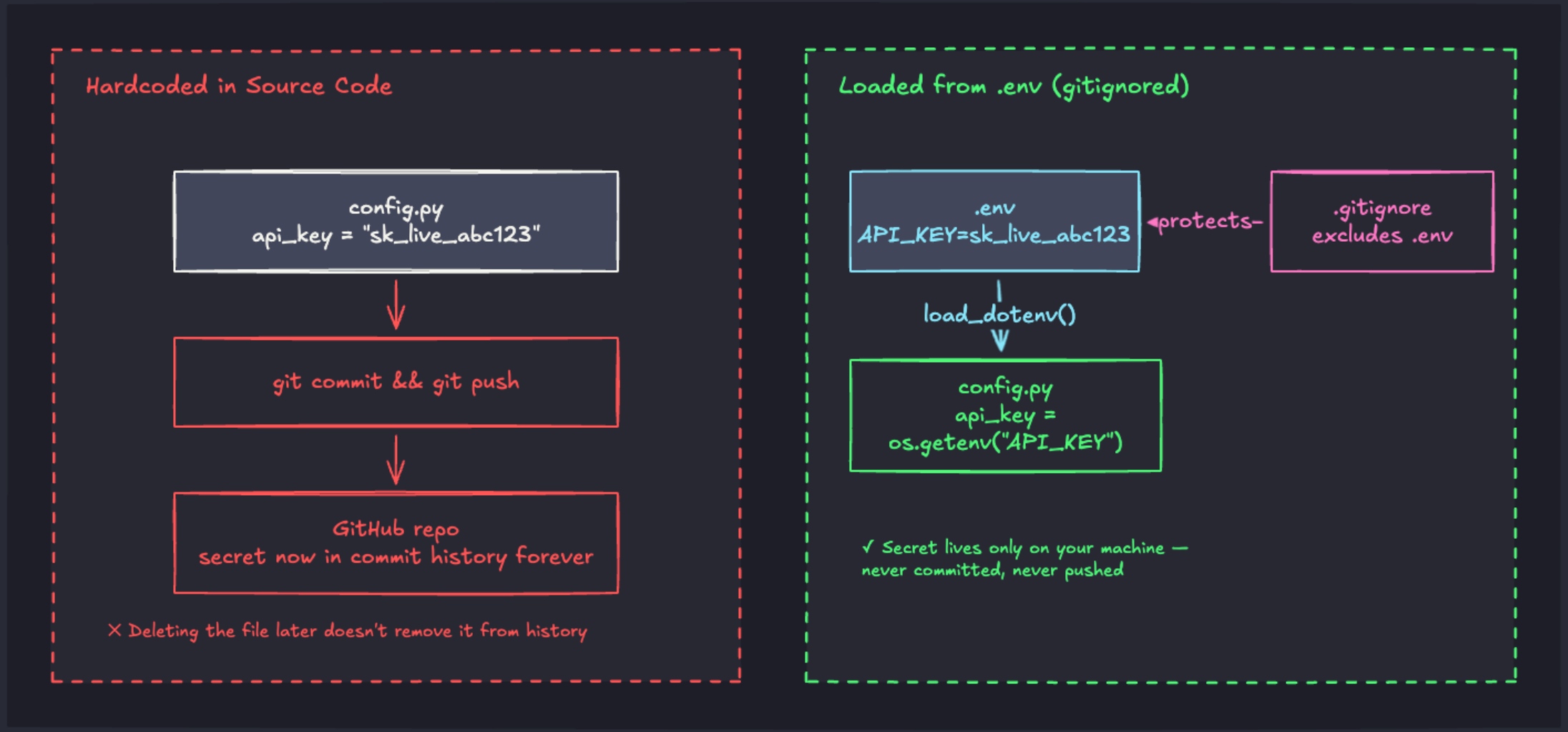
python-dotenv

The Problem: Config Baked Into Code

- API keys, passwords, and database URLs get typed straight into `.py` files
- That value is now identical in every environment — dev, staging, production
- `git push` sends it to GitHub, your teammates, and anyone who clones the repo
- Real leaked-key incidents happen constantly — bots scan public GitHub repos for exactly this

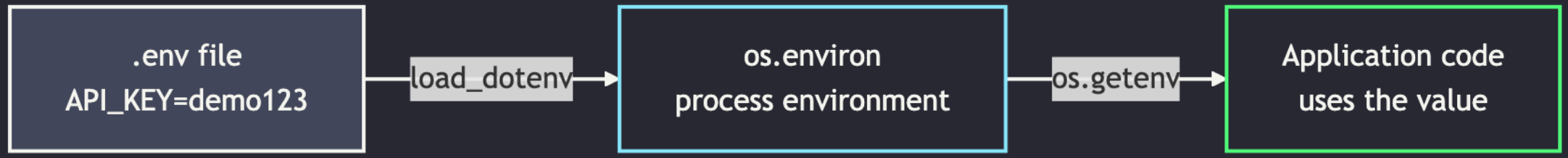
Hardcoded vs. `.env` -Based Config



What `python-dotenv` Does

- Reads `KEY=value` lines from a `.env` file in your project
- Loads each one into `os.environ` — the same place OS-level environment variables live
- Your application code just calls `os.getenv("KEY")` — it never knows or cares where the value came from
- Works identically whether the value came from `.env` locally or a real env var in production

How It Flows



Installation

```
# uv-managed project
uv add python-dotenv

# plain venv + pip
pip install python-dotenv
```

- `uv add` — adds `python-dotenv` to `pyproject.toml` and `uv.lock`
- `pip install` — installs into whatever venv is currently activated
- Either way: one dependency, no configuration required to get started

Demo: Create and Load a `.env`

```
# 1. Create a .env file in your project root
echo 'API_KEY=demo123' > .env
```

```
# 2. Load it at the top of your script
from dotenv import load_dotenv
load_dotenv()

# 3. Read the value like any environment variable
import os
print(os.getenv("API_KEY"))   # -> demo123
```

Demo: Missing Keys and Defaults

```
# Safe: returns None (or your default) if the key is missing
os.getenv("MISSING_KEY")            # -> None
os.getenv("MISSING_KEY", "fallback")  # -> "fallback"

# Unsafe: raises KeyError if the key is missing
os.environ["MISSING_KEY"]
```

- Prefer `os.getenv()` with a sensible default for optional config
- Use `os.environ[...]` only when a missing value should hard-fail the program

`.gitignore` and `.env.example`

```
# .gitignore
.env
```

```
# .env.example — committed, no real values
API_KEY=your-api-key-here
DATABASE_URL=postgresql://user:pass@localhost/db
```

- `.env` never gets committed — it holds real secrets
- `.env.example` **does** get committed — it documents what variables a teammate needs to set

Precedence & Gotchas

- **Real environment variables win** — `.env` never overrides an already-set OS env var, unless you pass `override=True`
- **Everything loads as a string** — `os.getenv("PORT")` returns `"8000"`, not `8000`; cast manually with `int(...)`
- **Order matters** — call `load_dotenv()` before any code that reads the variables
- **Path resolution** — `load_dotenv()` searches upward from the current directory by default; pass a path explicitly if needed

Cheat Sheet

Action	Code
Install	`uv add python-dotenv` or `pip install python-dotenv`
Load `.env`	`from dotenv import load_dotenv; load_dotenv()`
Read a value	`os.getenv("KEY")`
Read with default	`os.getenv("KEY", "default")`
Cast to int/bool	`int(os.getenv("PORT", "8000"))`
Ignore `.env` in git	add `.env` to `.gitignore`
Document variables	commit `.env.example`

References

Topic	Source
`python-dotenv` documentation	Kostrzewa, T. & contributors. (2026). python-dotenv. GitHub.
`python-dotenv` on PyPI	Python Packaging Index. (2026). python-dotenv. PyPI.
The Twelve-Factor App — Config	Wiggins, A. (2011–2017). III. Config. The Twelve-Factor App.

Recap & Practice

- Config and secrets belong in `.env`, not in source code
- `load_dotenv()` + `os.getenv()` is the whole API you need for most projects
- **Before you write a single line of code on your next assignment: add a `.env` and a `.env.example`**